DAYANANDA SAGAR UNIVERSITY

Devarakaggalahalli, Harohalli, Kanakapura Road, Ramanagara Dt,
Bengaluru-562112, Karnataka, India

SCHOOL OF
ENGINEERING

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

A Project Report On

Agent Frameworks for VLM based Medical Diagnosis

By

RATAN RAVICHANDRAN - ENG21AM0093

SAYLI PANKAJ BANDE - ENG21AM0112

Under the supervision of

Dr. Vinutha N

Associate Professor

Computer Science & Engineering (AI & ML)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

SCHOOL OF ENGINEERING
DAYANANDA SAGAR UNIVERSITY

(2024 – 2025)

DAYANANDA SAGAR UNIVERSITY

SCHOOL OF ENGINEERING

Department of Computer Science & Engineering

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

KUDLU GATE, BANGALORE – 560068

Karnataka, India

CERTIFICATE

This is to certify that the project entitled **"Agent Frameworks for VLM based Medical Diagnosis"** is carried out by **RATAN RAVICHANDRAN (ENG21AM0093) and SAYLI PANKAJ BANDE (ENG21AM0112)**, bonafide students of Bachelor of Technology in Computer Science and Engineering at the School of Engineering, Dayananda Sagar University, Bangalore, in partial fulfillment for the award of a degree in Bachelor of Technology in Computer Science and Engineering, during the year **2024 - 2025**.

Dr. Vinutha N	**Dr. Vinutha N**	**Dr. Jayavrinda Vrindavanam**
Associate Professor	Project Co-ordinator	Professor & Chairperson
Dept. of CSE (AIML)	Dept. of CSE (AIML)	Dept. of CSE (AIML)
School of Engineering	School of Engineering	School of Engineering
Dayananda Sagar University	Dayananda Sagar University	Dayananda Sagar University
Signature	Signature	Signature .....

Name of the Examiners:	Signature with date:
1.....	.....
2.....	.....
3.....	.....

DECLARATION

We, **RATAN RAVICHANDRAN (ENG21AM0093), SAYLI PANKAJ BANDE (ENG21AM0112)**, are students of the eighth semester B.Tech in Computer Science and Engineering (AI & ML) at the School of Engineering, Dayananda Sagar University. We hereby declare that the Major Project titled **"Agent Frameworks for VLM based Medical Diagnosis"** has been carried out by us and submitted in partial fulfillment for the award of a degree in **Bachelor of Technology** in **Computer Science and Engineering** during the academic year **2024–2025**.

Student: **Signature**

Name 1: RATAN RAVICHANDRAN
USN: ENG21AM0093

Name 2: SAYLI PANKAJ BANDE
USN: ENG21AM0112

Place: Bangalore
Date:

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of many individuals who have been responsible for the successful completion of this project work. First, we take this opportunity to express our sincere gratitude to School of Engineering& Technology, Dayananda Sagar University for providing us with a great opportunity to pursue our Bachelor's degree in this institution.

We would like to thank **Dr. Udaya Kumar Reddy K R, Dean, School of Engineering & Technology, Dayananda Sagar University** for his constant encouragement and expert advice. It is a matter of immense pleasure to express our sincere thanks to **Dr. Jayavrinda Vrindavanam, Department Chairman, Computer Science and Engineering (Artificial Intelligence and Machine Learning), Dayananda Sagar University**, for providing right academic guidance that made our task possible.

We would like to thank our guide **Dr. Vinutha N, Associate Professor, Dept. of Computer Science and Engineering of Artificial Intelligence and Machine Learning Dayananda Sagar University**, for sparing her valuable time to extend help in every step of our project work, which paved the way for smooth progress and fruitful culmination of the project.

We would like to thank our **Project Coordinator Dr. Vinutha N** as well as all the staff members of Computer Science and Engineering (AIML) for their support. We are also grateful to our family and friends who provided us with every requirement throughout the course. We would like to thank one and all who directly or indirectly helped us in the Project work

Contents

List of Figures

List of Tables

ABSTRACT

This project explores the integration of an AI agent framework with Vision-Language Models (VLMs) to enhance medical diagnostic processes. Current diagnostic systems face significant challenges, including fragmented data sources, limited adaptive learning capabilities, and the absence of intelligent frameworks for task coordination. Our solution leverages VLMs to analyze diverse medical data types, such as images and text, in real-time. By employing a chain-of-thought reasoning model, the AI agents provide reliable and explainable diagnostic insights, generate comprehensive reports, and offer real-time support to clinicians. This collaborative approach aims to streamline information exchange between healthcare professionals and AI systems, improving the speed and accuracy of diagnoses. The proposed framework addresses scalability issues by maintaining consistent performance across various clinical scenarios, ultimately enhancing patient care through faster and more precise decision-making.Using DeepEval metrics GEval, the framework has a score of 0.898, and perfect faithfulness at 1.000—exceeding high clinical thresholds. This research demonstrates the transformative potential of integrating advanced AI technologies into healthcare diagnostics, paving the way for more efficient and effective clinical practices.This integrated approach enhances over- all diagnostic performance, bridging the gap between general- purpose vision models and the specialized requirements of accurate radiographic diagnosis.

Chapter 1

INTRODUCTION

The rapid advancement of artificial intelligence (AI) technologies has opened new frontiers in healthcare, particularly in the realm of medical diagnostics. Traditional diagnostic systems often struggle with the integration of diverse data sources, such as medical images, electronic health records (EHRs), and clinical notes, which are crucial for comprehensive patient assessment. This fragmentation leads to inefficiencies and potential inaccuracies in diagnosis, underscoring the need for innovative solutions that can seamlessly integrate and analyze multimodal data.

- **The Promise of Vision-Language Models:** Vision–Language Models (VLMs) represent a significant breakthrough in AI, combining the capabilities of computer vision and natural language processing to understand and interpret complex datasets. These models are uniquely positioned to address the challenges faced by current diagnostic systems by providing a unified framework that can process both visual and textual information. The integration of VLMs into medical diagnostics promises to enhance the accuracy and speed of clinical decision-making, ultimately improving patient outcomes.

- **Proposed AI Agent Framework:** This project proposes an AI agent framework that leverages VLMs to automate and optimize medical image processing and analysis. The landscape of medical diagnostics has evolved dramatically over the past decade, with an exponential increase in the volume and complexity of patient data. Clinicians now face the daunting task of analyzing thousands of medical images, interpreting laboratory results, and reviewing extensive patient histories within limited time constraints. Our AI agent framework addresses these challenges by acting as an intelligent assistant that can process and synthesize vast amounts of medical information with remarkable speed and precision.

- **Intelligent Diagnostic Reasoning:** By implementing sophisticated algorithms and neural network architectures, these agents can identify subtle patterns and correlations that might escape human observation, thereby reducing the likelihood of misdiagnosis and improving clinical outcomes. The agents employ a chain-of-thought reasoning model to navigate complex medical scenarios—mimicking the diagnostic reasoning process of experienced clinicians

while maintaining transparency in their decision-making pathways. This approach not only facilitates real-time diagnostic support but also ensures that insights are explainable and reliable, fostering trust among healthcare professionals.

- **Real-Time Assistance & Reporting:** A key feature of this framework is its ability to generate detailed diagnostic reports and offer real-time assistance to clinicians. Our AI agents serve as a bridge between disparate systems, aggregating and contextualizing information from multiple sources to create comprehensive, standardized reports that enhance clinical communication. These reports include diagnostic findings, relevant medical literature, similar case studies, and treatment recommendations. During patient examinations, providers can consult the AI system for immediate feedback on observed symptoms, potential diagnoses, and appropriate follow-up tests—freeing them to focus on patient care rather than administrative tasks.

- **Adaptive Learning and Continuous Improvement:** Medical science is continuously advancing, with new research findings, treatment protocols, and diagnostic criteria emerging regularly. Static AI systems quickly become obsolete in this dynamic environment, potentially leading to outdated recommendations. Our framework incorporates adaptive learning mechanisms that enable the agents to refine their diagnostic capabilities based on clinician feedback and outcomes data. This continuous improvement cycle ensures that the system remains at the cutting edge of medical knowledge and can be tailored to the specific needs and protocols of individual healthcare institutions.

Chapter 2

PROBLEM DEFINITION AND OBJECTIVE

2.1 Problem Definition:

Current medical diagnostic systems face several interrelated challenges that limit their effectiveness and adoption:

- **Fragmented Data Integration:**
  - Diagnostic tools often operate in silos—medical imaging platforms, laboratory information systems, EHRs, and clinical-note repositories rarely communicate seamlessly.
  - Inconsistent data formats and lack of standardized interfaces lead to information loss, redundancies, and manual reconciliation efforts.
- **Absence of Adaptive Learning:**
  - Most AI-based tools are "static" once deployed, unable to refine their performance based on real-world feedback.
  - Without mechanisms to learn from expert corrections or evolving best practices, their recommendations can become outdated.
- **Lack of Coordinated Multi-Agent Frameworks:**
  - Current solutions rarely orchestrate multiple specialized models (e.g., separate modules for image analysis, text mining, lab-value interpretation).
  - This siloed approach prevents end-to-end automation of the diagnostic workflow and burdens clinicians with stitching together disparate outputs.

- **Need for Real-Time, Explainable Support:**
  - Clinicians require on-demand insights during patient interactions—but existing systems either run offline or produce opaque "black-box" recommendations.
  - The lack of real-time, transparent reasoning undermines trust and hinders uptake in high-stakes clinical settings.
- **Scalability and Consistency Across Contexts:**
  - Diagnostic accuracy often degrades when systems are moved between hospitals, specialties, or patient populations.
  - A robust solution must maintain performance despite variations in imaging devices, lab equipment, and demographic factors.

2.2 Objective:

To address these gaps, this project aims to design and implement an AI-agent framework that:

- **Integrates Multimodal Data:** Seamlessly ingests and harmonizes medical images, EHR entries, lab reports, and clinical narratives into a unified analysis pipeline.
- **Employs Adaptive Learning Mechanisms:** Continuously refines its diagnostic models based on clinician feedback, outcome data, and emerging medical guidelines.
- **Coordinates Specialized Sub-Agents:** Orchestrates dedicated modules for vision tasks, language understanding, and numerical lab-value interpretation into a coherent end-to-end diagnostic assistant.
- **Provides Explainable, Real-Time Decision Support:** Utilizes chain-of-thought reasoning to trace its analysis steps and delivers immediate, interpretable insights at the point of care.
- **Ensures Scalability and Robustness:** Leverages transfer learning and domain-adaptation strategies to preserve accuracy across institutions, imaging modalities, and patient cohorts.

2.3 Approach Overview:

- **Agent-Based Architecture:** Deploy modular agents—each specialized for a data type—under a central controller that manages information flow and aggregates their outputs into a consolidated report.
- **Vision-Language Model (VLM) Core:** Use state-of-the-art VLMs to jointly interpret images and text, enabling cross-modal feature fusion and richer pattern recognition.
- **Feedback-Driven Refinement:** Implement a clinician-in-the-loop mechanism where expert annotations trigger automated model fine-tuning and update the system's knowledge base.

2.4 Expected Impact:

- **Timely, Accurate Diagnoses:** Reduced time to diagnosis and fewer missed or incorrect findings through automated pattern detection and cross-validation across data sources.
- **Enhanced Clinical Workflow:** Alleviation of manual data aggregation tasks, allowing healthcare providers to focus on patient interaction and decision making.
- **Broader Adoption of AI Tools:** Transparent reasoning and adaptability will foster greater confidence among clinicians and support regulatory approval processes.

Chapter 3

LITERATURE SURVEY

The foundational work by Huang et al., "Artificial Intelligent Agent Architecture and Clinical Decision-Making in the Healthcare Sector," presents a comprehensive agent–environment model in which autonomous software agents observe patient data streams—from EHRs and imaging devices—apply decision logic, and enact recommendations within clinical workflows. This paper delineates key agent properties—reactivity, proactiveness, and social ability—and maps them to use cases such as continuous vital-sign monitoring and automated triage, demonstrating both theoretical architectures and initial prototype integrations with hospital information systems. Through a detailed analysis, the authors show how modular agent components reduce clinician workload and improve diagnostic throughput, laying the groundwork for more adaptive and intelligent clinical decision support platforms. [1]

Wu et al. introduce AutoGen, an open-source multi-agent framework that enables developers to compose conversable agents which coordinate via natural-language or code-based protocols to accomplish complex tasks. By integrating Retrieval-Augmented Question Answering, AutoGen agents dynamically update their conversational context based on user input and retrieved knowledge, yielding superior performance over single-agent baselines in domains ranging from mathematical problem solving to interactive coding assistance. The framework's flexibility in defining agent behaviors through both declarative JSON and prompt templates underscores its applicability to healthcare scenarios where multiple specialized agents—such as symptom-gatherers and differential-diagnosers—must collaborate seamlessly. [2]

"Agent Hospital: A Simulacrum of Hospital with Evolvable Medical Agents" presents an innovative virtual environment where AI-driven doctor agents learn to diagnose and treat simulated patients using the MedAgent-Zero trial-and-error strategy. Within this simulacrum, agents repeatedly interact with thousands of patient-agent cases, storing both successful treatments and failure-derived rules in experience bases, which leads to marked improvements in diagnostic accuracy across 32 medical departments and 339 diseases. The study demonstrates that such automated experiential learning achieves over 91 percent accuracy on the MedQA benchmark,

validating the scalability and real-world applicability of evolvable medical agents. [3]

The survey "Artificial Intelligence in Medical Diagnostics: Current Trends and Future Directions" reviews how chain-of-thought reasoning and explainable AI (XAI) techniques enhance diagnostic systems by making their decision processes transparent and trustworthy. It emphasizes that chaining multiple specialized AI agents—each handling tasks like image segmentation, feature extraction, and clinical knowledge retrieval—can yield higher interpretability and diagnostic accuracy, though at the cost of increased computational complexity and resource requirements. The authors conclude by outlining future research avenues, including federated learning for privacy-preserving diagnostics and tighter human-in-the-loop feedback loops to continuously refine agent performance. [4]

Chen et al. propose "Reinforcing Clinical Decision Support through Multi-Agent Systems and Ethical AI Governance," a framework that employs modular laboratory-analysis, vitals-interpretation, contextual-reasoning, and validation agents to drive patient-specific predictions in intensive care settings. By embedding ethical AI principles—Autonomy, Fairness, Accountability—into the agent orchestration layer, the system produces transparent and verifiable decision trails, achieving notable improvements in both interpretability and trust in high-stakes environments. [5]

In "Autonomous Artificial Intelligence Agents for Clinical Decision Making in Oncology," the authors develop an oncology-focused multi-agent pipeline combining radiology image analysis, genomic data processing, guideline retrieval, and report generation agents. Evaluated across simulated oncology patient scenarios, the system selects appropriate domain tools 97 percent of the time, achieves 93.6 percent accuracy in diagnostic conclusions, and consistently references relevant literature (82.5 percent) within its rationales. This study demonstrates that LLM-enabled autonomous agents can function as specialist clinical assistants, offering patient-tailored recommendations while simplifying regulatory validation by isolating each component for individual approval. [6]

Chapter 4

METHODOLOGY

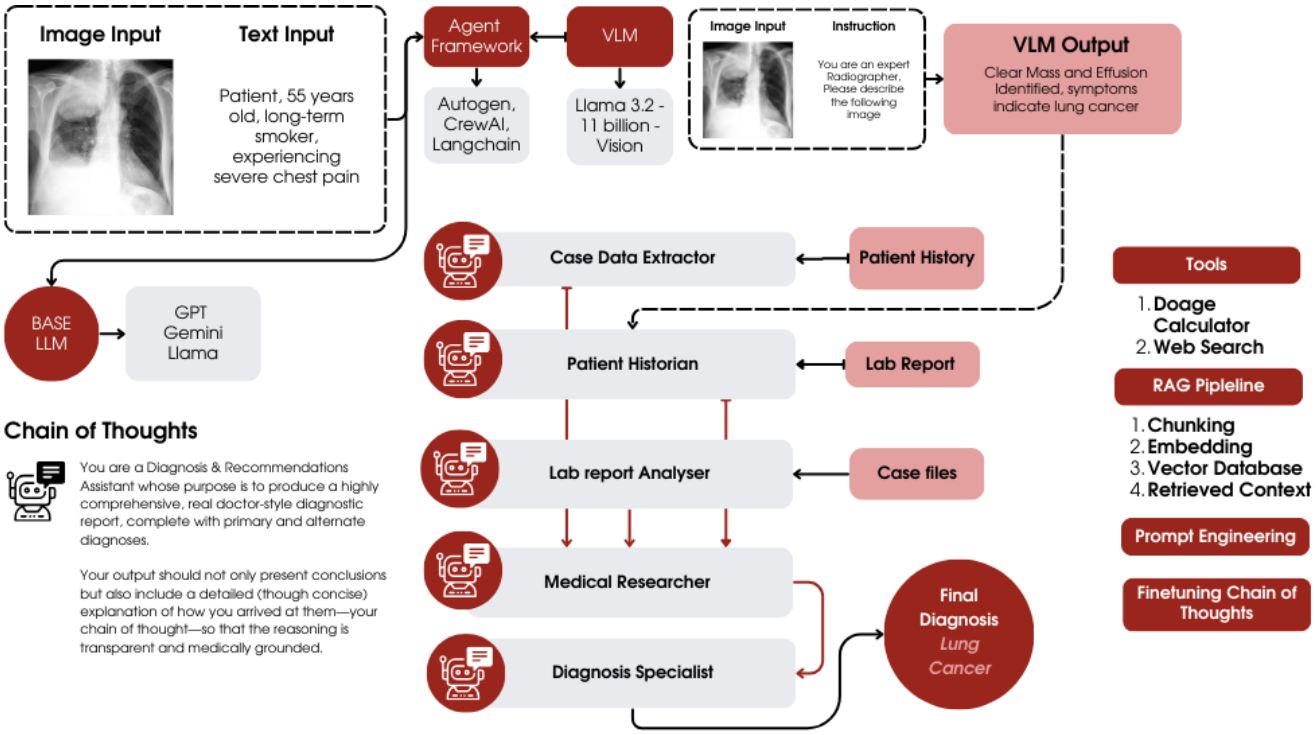


Figure 4.1: Architecture Workflow

4.1 Architectural Overview of the Multi-Agent Diagnostic Framework

Framework Design:

As seen in the figure above, the system is designed as a collaborative multi-agent framework that mirrors a real-world medical team consulting on a diagnosis. At its core, several specialized AI agents (each powered by Large Language Models, LLMs) work together, alongside external tools and a shared knowledge base, to analyze patient data and propose diagnoses. The architecture

consists of three primary layers: (1) an input processing layer for ingesting multimodal patient data (text and images) and encoding it into a unified knowledge store, (2) a multi-agent reasoning layer where each agent contributes expert analysis in a specific domain, and (3) an output synthesis layer that compiles the agents' findings into a final structured report. A highlevel view of this design is illustrated by analogous frameworks in recent research, where groups of LLMbased agents collaborate on complex reasoning tasks while sharing a common context or memory . This design is particularly apt for medicine: clinical decision-making is inherently multi-faceted and collaborative, often involving input from different specialists (e.g. primary doctors, specialists, ethicists) and data types (textual records, lab results, medical imaging) . By structuring the AI into specialized agents with a shared context, our framework emulates this collaborative process in a systematic, reproducible way.

In this architecture, all input data – including textual records and medical images – is ingested and transformed into machine-interpretable forms that are stored in a vector database. The vector database serves as the system's long-term memory and knowledge repository, enabling a retrieval-augmented generation (RAG) approach to AI reasoning. Each agent can fetch relevant information from this repository as needed, rather than relying solely on the LLM's internal memory. The agents communicate through an orchestrated conversation, sharing their intermediate findings via a common context so that each agent's output becomes part of the input for the others. An agent orchestration framework manages this multi-agent dialogue, ensuring that each agent speaks in turn, adheres to its role, and has access to the accumulated context (including retrieved facts and prior agent outputs). Overall, the architecture enables a chain of analytical steps: from data ingestion, to distributed expert analysis, to aggregated diagnostic reasoning. This chain-of-thought style process – where reasoning is broken into discrete, interdependent steps – helps maintain coherence and thoroughness in complex cases . In the following sections, we detail how different input modalities are handled, how information is stored and retrieved, the specific roles of each agent, and how the agents are orchestrated to produce a final diagnosis and treatment plan.

4.2 Input Modalities: Text, Images, and Multimodal Context

- **Textual Input:** Most patient information arrives as unstructured or semi-structured text (e.g., patient history narratives, physician notes, case reports, or lab result printouts). The system first uses a file ingestion utility (a `FileReadTool`) to read raw text from files or electronic health record (EHR) exports. Any text in PDF or scanned formats is run through an OCR/parser if needed, converting it into plain text. The text is then preprocessed: cleaned of irrelevant formatting, segmented into semantically coherent chunks (e.g., a paragraph per clinical note or a table row per lab test), and enriched with metadata (such as source type or date). Each text chunk is vector-encoded into an embedding representation and stored in the vector database, allowing later semantic similarity search. Key textual data—for instance, a

summary of the patient's chief complaint or a list of active medications—may also be extracted explicitly by an agent (the Case Data Extractor) and placed into the shared context so that critical facts are immediately visible to all agents.

- **Image Input:** The system can also handle medical images (such as radiology scans, pathology slides, or clinical photographs) as part of the diagnostic context. Images are processed by specialized sub-components integrating computer vision models. For example, an imaging file is passed to a medical vision model (which could be a chest X-ray classifier, an MRI analyzer, or a multimodal LLM with vision capabilities) that outputs a textual description or classification of the image's contents (e.g., "Chest X-ray shows bilateral patchy infiltrates suggestive of pneumonia"). This textual description is then embedded as a vector and stored in the knowledge base. In some cases, structured data might be extracted (for instance, a model detecting a tumor will output the tumor's characteristics). The original image can also be retained as a reference, but all reasoning agents consume the textual interpretation of the image rather than the raw image. By converting images into natural language observations, the framework ensures that visual data is integrated into the same context as textual data, creating a unified multimodal representation in the vector database—allowing a doctor's note and an X-ray study to be retrieved and considered side by side when relevant.

- **Multimodal Context Integration:** In complex cases, inputs include combinations of text (clinical notes, lab reports, etc.) and images (e.g., radiology). The framework handles such multimodal contexts by processing each modality through its respective pipeline (as above) and then merging the results in the shared context. Concretely, after preprocessing, all extracted textual information (from text documents or image captions) is indexed in the vector database under the current case ID. Agents do not need to handle modalities separately; they query the common vector store or read from the conversation memory, retrieving information from either text or image origin as needed. For example, if the Diagnostic Specialist queries the knowledge base for "evidence of infection," the system may return a snippet from lab report text (e.g., elevated white cell count) as well as a description from an imaging study (e.g., an X-ray indicating pneumonia). By leveraging a shared embedded context, the system transcends data format boundaries—everything is represented in the same semantic vector space, enabling seamless cross-referencing between modalities. Overall, integrated handling of text and images is crucial, as modern medical diagnosis often relies on correlating narrative reports with imaging findings.

4.3 Knowledge Base and Retrieval-Augmented Generation (RAG) via Vector Database

Central to the framework is a vector database that powers a Retrieval-Augmented Generation mechanism. The vector database (a high-dimensional index of embedding vectors) serves as the

unified knowledge store for both patient-specific data and external medical references. By converting all inputs and reference documents into embedding vectors and storing them, the system enables semantic retrieval of information that is relevant to the task at hand. In practice, this means that when an agent formulates a query or requires additional facts (for example, "Has the patient had any prior heart conditions?" or "What do the latest guidelines say about treating this presentation?"), the system can search the vector database for contextually similar content and supply those snippets to the agent.

- **Role of the Vector Database:**
  The vector DB underpins the system's memory and knowledge capabilities. It contains two broad classes of data:
  - *Patient Case Data:* Processed history, lab findings, imaging results for the case in question.
  - *External Medical Knowledge:* Textbook excerpts, medical journal articles, past case reports, clinical guidelines, etc.

  All of these are stored as embeddings, allowing the system to perform similarity searches in lieu of exact keyword matching. This is particularly powerful in medical applications, as the wording of a doctor's note might differ from the phrasing in literature—semantic search bridges that gap. For instance, an agent could retrieve a relevant snippet about "chest pain causes" even if the patient history used colloquial descriptions. By using vector search, the system finds information despite vocabulary differences or incomplete keywords.

- **On-Demand Knowledge Retrieval:**
  The vector database ensures that LLM agents always have access to information beyond their fixed training data, including up-to-date research or patient-specific details. This grounding in retrieved evidence mitigates hallucination and outdated knowledge, significantly improving the accuracy and relevance of the AI's outputs.

4.3.1 Retrieval-Augmented Generation Workflow

Whenever an agent needs information, it invokes a retrieval step via the `MDXSearchTool`, which queries the vector database:

1. The agent embeds its natural-language query (e.g., "Find recent case reports similar to this patient's presentation").
2. A nearest-neighbor search returns top-matching passages (e.g., a journal paragraph, a textbook snippet).
3. Retrieved texts are tagged with source metadata and appended to the agent's prompt context.

4. The agent generates its response, directly informed by these retrieved snippets.

Additional RAG enhancements include hybrid search (combining dense vectors with keyword matching) and relevance re-ranking. All retrieved chunks remain in the conversation memory for subsequent agents to reference, ensuring that every claim or recommendation is grounded in real data.

4.3.2 Preprocessing of Data Sources for RAG

Each data source is preprocessed before embedding:

- **Patient History Documents:** Segmented into semantic chunks (e.g. "History of Present Illness", "Family History"), optionally summarized via an LLM, then embedded with metadata (patient ID, section type) for filtered searches.
- **Case Reports and Medical Literature:** Curated corpus split along natural boundaries (paragraphs, headings), domain-specific embeddings applied, and diversity ensured across conditions and treatments to form the reference library.
- **Laboratory and Imaging Reports:** Structured tables converted into text summaries (e.g. "Blood test: WBC 15×10^9/L (high)"), imaging findings described textually, then embedded to allow retrieval of specific lab values or image observations without re-loading full reports.

4.4 Data Sources and Preprocessing

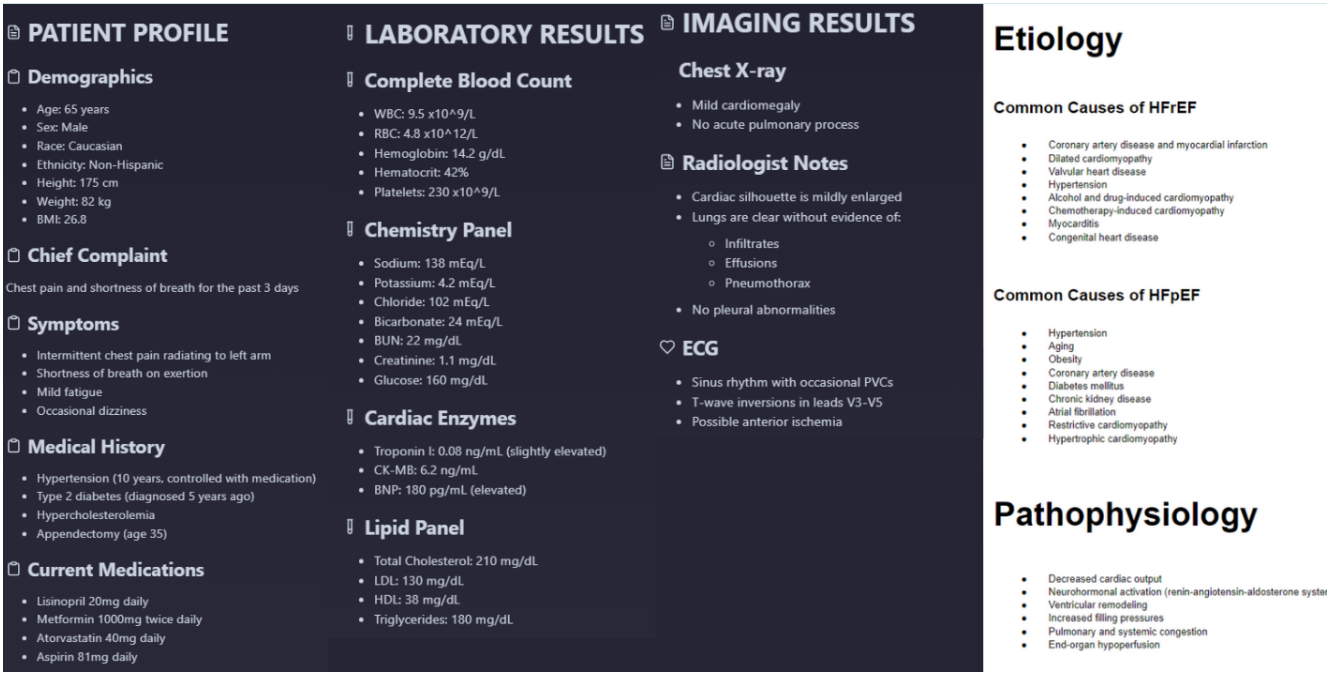


Figure 4.2: Patient and Medical Data

4.4.1 Patient History Records

Patient history is the narrative of the patient's medical background and current complaints. It includes the history of present illness, past medical and surgical history, medications, allergies, family and social history, and other notes recorded by healthcare providers. In our framework, patient history documents (which may come as free-text clinical notes or referral letters) are given special attention during preprocessing because they contain many clues essential for diagnosis.

Every patient history document is parsed and cleaned to remove extraneous information (headers, signatures, etc.). We then apply a combination of rule-based and LLM-based techniques to extract structured elements from the narrative. For instance, the system can identify sections like "Chief Complaint" or "Past Medical History" by looking for standard section headings or keywords. If found, it will isolate those sections for focused analysis. An agent—the Patient Historian—is tasked with further summarizing and analyzing this text (more details on the agent below). Additionally, the entire history (or its segmented parts) is embedded into the vector database. The embedding process uses a medical-specific language model to capture clinical terminology effectively. The result is that the patient's history is readily searchable: e.g., another agent can later retrieve "history of diabetes" if the patient had such a note in their past records.

We also incorporate a step of contextual abstraction for history data. This means the Patient Historian (or a preprocessing routine) will distill the raw text into a concise summary of facts: key symptoms, timelines, and relevant conditions. This summary is placed into the shared conversation context so that it's immediately available without retrieval. Meanwhile, finer details remain accessible via vector search if needed. By doing this, we ensure that critical historical facts (e.g., "patient has a 10-year history of hypertension" or "family history of autoimmune disease") are not overlooked. In essence, patient history data is treated both as direct input to be summarized and as indexed knowledge for specific detail lookup. This two-pronged approach allows the system to maintain a broad understanding of the patient's background while also being able to drill down on particular points if the diagnostic reasoning calls for it.

4.4.2 Case Reports and Medical Literature

Case reports and related medical literature form the knowledge backbone that the AI agents draw upon to contextualize the patient's case. These include documented case studies of past patients, research articles, clinical trial reports, treatment guidelines, and textbook knowledge. Since no AI (especially one focused on diagnosis) should operate in isolation from established medical knowledge, we built an extensive corpus of such references and integrated it via the vector database.

Prior to running the diagnostic agents, this corpus is preprocessed offline. Each document is divided into logical sections (for example, Background, Case Presentation, Discussion, etc.). Domain-specific stopwords (common medical terms that don't aid in distinguishing content) are removed or down-weighted in embeddings to improve retrieval focus. Each section of each document is then encoded as an embedding and stored in the vector DB, tagged with metadata

like source title, publication date, and document type. We also include embeddings of figures or tables via captions or descriptions, so that important data in charts (like a survival curve or a treatment flowchart) isn't lost.

When the system is running, the Medical Researcher agent primarily interfaces with this data source. Upon request (either automatically when a case begins, or dynamically when needed), the agent uses the MDXSearchTool to query this corpus. For example, if the patient is a 45-year-old male with chest pain and ambiguous lab results, the researcher might search for "case report middle-aged male chest pain normal troponin atypical". The vector DB returns semantically similar entries, perhaps an excerpt from a journal about atypical heart attack presentations with normal initial biomarkers. Those snippets are then presented to the researcher agent's LLM, which can read and synthesize them into useful knowledge—for instance, noting that "some myocardial infarctions can present with normal troponin initially." The agent may include citations or source identifiers in its output (though for the final report we usually convert these into a readable form rather than raw URLs).

Importantly, the retrieval system doesn't just pull any text—it often uses a re-ranking step to ensure the most relevant and high-quality information is prioritized. For example, if five documents are retrieved, a secondary model (like a cross-encoder) evaluates which of those most directly addresses the query, and the top two or three are then used by the LLM agent. This prevents distractions by tangential facts. All retrieved content from case reports and literature is kept as part of the conversation context for subsequent agents, so that if the Diagnostic Specialist later needs to justify a diagnosis, it can leverage those same literature snippets as supporting evidence.

One additional preprocessing measure is knowledge updating. Medical knowledge evolves rapidly, so our system periodically updates the literature corpus by ingesting new guidelines or recent high-impact case studies. We use the SerperDevTool (a web search API integration) and ScrapeWebsiteTool for this purpose during development: they allow the system (or developers) to fetch the latest information from the web (such as new research or drug updates) and add it to the vector store. However, during an active diagnostic session, internet access might be restricted for safety; in such cases, the system relies on the curated knowledge it has, which we keep as current as possible. This combination of a static vetted knowledge base and the ability to perform real-time searches when allowed gives the framework both reliability and adaptability.

4.4.3 Laboratory Results and Reports

Lab reports provide quantitative data (and sometimes qualitative observations) about the patient's condition—for example, blood test results, urine analysis, pathology findings, or radiology reports. These are critical for diagnosing many conditions, and our system treats them as a distinct data source with specialized handling.

When a lab report is provided (often as text or a PDF), the Case Data Extractor or Lab Interpreter agent parses it. Numerical lab results are extracted into key–value pairs (e.g., Hemoglobin: 9.5g/dL). We cross-reference these with normal ranges (either provided in the report or via an

internal reference database) to annotate each result as high, low, or normal. This annotation may use simple rules or a brief LLM prompt (e.g., "interpret these lab values and note which are abnormal"). The outcome is a structured summary of the labs. This summary is stored in the vector database and also presented in the context so that all agents are aware of key lab abnormalities.

For more complex reports, such as pathology or radiology impressions (e.g., "MRI Brain: There is a 2 cm lesion in the left frontal lobe suggestive of meningioma."), the descriptive impression is extracted in full and embedded. The Lab Interpreter agent then reads these impressions and may simplify or highlight the conclusion (e.g., "MRI suggests a left frontal meningioma"). If actual images are provided (like DICOM files), a vision model produces a textual description that is handled similarly.

All lab and imaging data, once processed, is thus available for retrieval. If the Diagnostic Specialist queries "evidence of anemia," the vector search might return the lab summary noting low hemoglobin. Conversely, the Medical Researcher agent could query the knowledge base with an abnormal lab as context (e.g., "causes of elevated AST with normal ALT") to explore underlying conditions.

Finally, lab and imaging data also feed into the Ethics Advisor agent's considerations. For example, if a genetic test or sensitive diagnostic procedure is present, the Ethics Advisor might flag it (e.g., implications for family members or insurance). The preprocessing stage tags such results with their type and any known ethical considerations (like if a result indicates a reportable disease). This way, the Ethics Advisor can easily retrieve and review those points when formulating its advice.

4.5 Specialized Agents and Their Roles

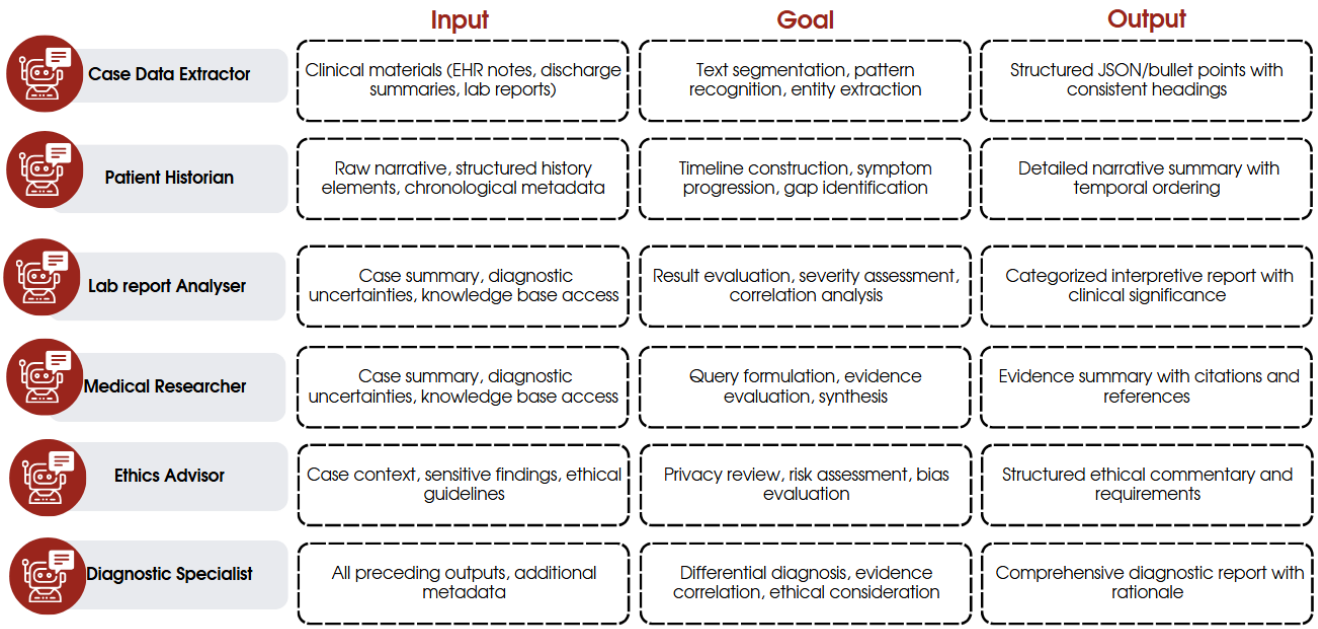


Figure 4.3: Agents and their roles

- **Case Data Extractor:**
  - **Inputs:**

    The Case Data Extractor ingests all unstructured and semi-structured clinical materials associated with a new patient case. This includes free-text notes from electronic health records (EHR), discharge summaries, referral letters, raw laboratory reports (as text tables or PDFs), metadata and captions from medical images, and any patient-provided documents. It can also receive pre-existing structured data (e.g. demographic fields) and descriptions generated by upstream image-analysis modules. Together, these inputs encompass narrative history, quantitative test values, and descriptive findings.
  - **Processing:** Using an LLM guided by a layered prompting strategy, the agent first segments lengthy text into manageable chunks, then applies pattern-based recognition and named-entity extraction to identify clinical concepts—symptoms, durations, diagnoses, test names, values, units, and normal reference ranges. It iteratively refines its output by cross-checking extracted values against standard thresholds (e.g. flagging values outside normal limits) and by merging synonymous terms (for instance, mapping "shortness of breath" and "dyspnea"). The extractor uses chain-of-thought prompts to ensure completeness, prompting itself for missing categories until every section of the case file—history, labs, imaging, medications, allergies—has been systematically addressed.

- **Output:** The final deliverable is a machine-readable structured summary (often formatted as JSON or standardized bullet points) that organizes key patient facts under consistent headings (Presenting Complaints, Past History, Laboratory Findings, Imaging Results, etc.). Each entry includes the data point, its value, unit, date, and a normality flag when applicable. This summary is injected into the shared conversation context for downstream agents and persisted in a vector database with appropriate semantic tags to support rapid retrieval and indexing.

- **Patient Historian:**
  - **Inputs:** The Patient Historian receives the raw narrative of the patient's medical background—EHR notes, past discharge summaries, social history entries, family history logs—and the structured history elements extracted by the Case Data Extractor. It also has access to any chronological metadata (timestamps of encounters) and demographic fields such as age, gender, occupation, and lifestyle factors.
  - **Processing:** Guided by prompts to construct a coherent timeline, the agent synthesizes these inputs into a seamless narrative, identifying onset dates, durations, and progression of key diagnoses or symptoms. It highlights chronic conditions (e.g., diabetes, hypertension), prior interventions (surgeries, hospitalizations), medication regimens, and relevant social determinants (occupational exposures, habits). The agent applies clinical reasoning to infer which historical details bear on current presentation—flagging gaps (such as missing smoking history in a respiratory case) and inconsistencies—and uses follow-up prompts to ensure no critical context is omitted.
  - **Output:** The output is a detailed, prose-style summary of the patient's history, typically spanning several paragraphs. It outlines major health events in temporal order, emphasizes risk factors and comorbidities, and calls out any uncertainties or missing data. This narrative is stored both in the conversation context for other agents and in the knowledge base for future cross-case retrieval.

- **Lab Interpreter:**
  - **Inputs:** The Lab Interpreter is furnished with all diagnostic test data: numerical laboratory panels (complete blood counts, metabolic panels), reference ranges, descriptive radiology or pathology reports, and any preliminary image-analysis annotations. It also receives the structured lab entries from the Case Data Extractor to ensure no value is overlooked.
  - **Processing:** Applying clinical pathophysiology knowledge embedded in the LLM, the agent evaluates each abnormal result for its severity and likely clinical implications. It computes derived indices (e.g., estimated glomerular filtration rate, neutrophil-to-lymphocyte ratio) when relevant and correlates lab abnormalities with presenting symptoms (e.g., anemia explaining fatigue). For imaging descriptions, it translates radiologic

terminology into actionable insights, noting findings that warrant further investigation. It uses internal consistency checks—such as verifying that elevated inflammatory markers align with suspected infections—to refine its interpretations.

- **Output:** A structured interpretive report is generated, often organized by category (Hematology, Chemistry, Imaging), with each entry providing the test name, result, clinical significance, and recommended next steps. This narrative output is appended to the case context and guides both the Diagnostic Specialist's reasoning and the Medical Researcher's literature queries.

- **Medical Researcher:**

  - **Inputs:** The Medical Researcher takes as its starting point the consolidated case summary (history, labs, imaging interpretations) and any explicit questions or diagnostic uncertainties raised by earlier agents. It also has programmatic access to an internal vector-indexed literature repository and external search tools for the latest guidelines or case reports.

  - **Processing:** The agent formulates precise search queries to both the internal knowledge base and external medical databases, retrieves relevant articles and guidelines, and uses web-scraping as needed to extract full text. It evaluates source credibility, cross-validates findings across multiple references, and synthesizes key evidence points—highlighting diagnostic criteria, differential-diagnosis frequencies, and management recommendations. Iterative refinement ensures that emerging hypotheses are rigorously tested against up-to-date clinical literature.

  - **Output:** A concise evidence synthesis is produced, comprising bullet-pointed or short-paragraph summaries of each relevant finding, complete with citations or guideline references. This report informs the Diagnostic Specialist and flags any regulatory or best-practice considerations for the Ethics Advisor.

- **Ethics Advisor:**

  - **Inputs:** This agent monitors the entire case context—patient identifiers, sensitive findings (e.g., genetic or infectious disease results), proposed diagnostics or interventions—and any retrieved ethical guidelines or regulations.

  - **Processing:** Using a prompt framework grounded in medical ethics principles (autonomy, beneficence, non-maleficence, justice), the agent reviews privacy implications of data sharing, assesses the necessity and risks of invasive procedures, checks for potential bias in demographic-based reasoning, and evaluates patients' capacity for informed consent. It cross-references relevant professional codes and public health laws to identify any reporting obligations or equity concerns.

  - **Output:** A structured ethical commentary is generated, outlining confidentiality requirements, informed-consent protocols, risk-benefit analyses, bias mitigation strategies, and legal or public-health mandates. These considerations are appended to the case context for incorporation into the final plan.

- **Diagnostic Specialist:**

  - **Inputs:** The Diagnostic Specialist has at its disposal all preceding outputs: the structured data summary, historical narrative, lab interpretations, research synthesis, and ethical commentary, along with any additional metadata (e.g., vital signs, demographics).
  - **Processing:** Employing a transparent chain-of-thought approach, the agent restates the clinical problem, generates a ranked differential diagnosis, correlates each possibility with the extracted data and literature evidence, and applies ethical constraints. It weighs laboratory and imaging findings against historical risk factors, references guideline-based recommendations, and performs necessary calculations or rule-outs. The specialist iteratively refines its reasoning in light of ethical guidance, ensuring that proposed actions respect patient autonomy and public-health requirements.
  - **Output:** A comprehensive diagnostic report is drafted, organized into sections for Diagnostic Conclusion, Differential Diagnosis, Rationale, Recommended Tests, Treatment Plan, Ethical Considerations, and Follow-Up. Each section presents clear, evidence-based reasoning and specific, actionable recommendations, formatted for clinical use and patient communication.

4.6 Agent Collaboration Workflow: Sequential Coordination and Information Flow

The multi-agent framework operates through a carefully orchestrated, step-by-step workflow that mirrors a collaborative case discussion in clinical practice. Each agent contributes in a fixed sequence, and their messages accumulate in a shared conversational context managed by the orchestrator. This ensures ordered contributions, logical information flow, and a cohesive diagnostic result.

- **Case Initialization:**
  When a new case is introduced, the orchestrator seeds the session with:
  - A brief *system message* outlining goals, constraints, and reminders to stay within each agent's expertise.
  - Any *cached background knowledge* pre-loaded for the scenario.
  - A recap of each agent's role.

  At this point, the raw input data is available but not yet processed.

- **Stage 1 – Data Extraction (Case Data Extractor):**
  The orchestrator designates the *Case Data Extractor* as the first speaker. It reads the raw inputs and outputs a structured summary of key facts. This summary is inserted into the shared context, establishing the factual foundation for all subsequent agents.

- **Stage 2 – Historical Analysis (Patient Historian):**
  The *Patient Historian* reviews the Case Extractor's summary and the raw history text, composing a detailed narrative of the patient's background. This enriched history is appended to the context, so the conversation now contains both bullet-point facts and a narrative history.

- **Stage 3 – Lab Interpretation (Lab Interpreter):**
  Next, the *Lab Interpreter* references raw lab values, highlighted abnormalities, and any imaging descriptions. It contributes a clinical interpretation of these findings, which the orchestrator appends to the context. At this point, the conversation contains facts, history, and interpreted diagnostics.

- **Stage 4 – External Research (Medical Researcher):**
  The *Medical Researcher* reads the accumulated context and decides what external knowledge to retrieve. During its turn it may:
  - Issue a `MDXSearchTool` query to the vector database (e.g. "TB reactivation hemoptysis").
  - Receive retrieved passages as an *observation* and refine the query if necessary.
  - Compile a summary of external evidence and references.

The orchestrator inserts this summary into the context for all agents to use.

- **Stage 5 – Ethical Oversight (Ethics Advisor):**
  The *Ethics Advisor* reviews the entire shared context—raw data, history, labs, external evidence—and outputs a guidance message focusing on privacy, consent, bias, harm–benefit balance, and regulatory compliance. This message is appended, completing the perspective set.

- **Stage 6 – Diagnostic Reasoning (Diagnostic Specialist):**
  Finally, the *Diagnostic Specialist* receives full context and performs a chain-of-thought analysis:

  1. Synthesises the core problem statement.
  2. Generates and weighs the differential diagnosis against labs, imaging, and history.
  3. Integrates evidence from the Medical Researcher.
  4. Applies ethical guidance.
  5. Produces the final diagnosis, management plan, and rationale.

  The orchestrator captures this as the culmination of the chat.

- **Conversation Closure:**
  One complete cycle usually suffices. If ambiguities remain (e.g. the Diagnostic Specialist signals uncertainty), the orchestrator may coordinate an additional mini-round—asking the Medical Researcher for targeted evidence or the Lab Interpreter for clarification—up to a configured maximum of three cycles to prevent infinite loops.

- **Shared Context as Memory:**
  All agents *implicitly* receive previous messages via the GroupChat context, eliminating explicit message-passing logic. Short-term memory (recent messages) and long-term memory (vector store queries) coexist, enabling agents to recall or retrieve details beyond the immediate context window.

- **Sequential vs. Parallel Execution:**
  A sequential order is chosen for clarity and reproducibility, mirroring structured medical case conferences.

- **Iterative Refinement and Termination:**
  A heuristic uncertainty detector can trigger extra rounds for clarification, but the session terminates once the Diagnostic Specialist's output covers diagnosis, plan, ethics, and evidence comprehensively. The orchestrator then compiles the conversation into the final report.

- **Collaborative Chain-of-Thought:**
  By dividing cognitive load across specialized agents and linking their outputs in a single conversational chain, the framework achieves deep, transparent reasoning akin to multidisciplinary teamwork in clinical practice.

4.7 Orchestration Framework and Tools Integration

4.7.1 Agent Coordination and Role Enforcement

The multi-agent diagnostic system is built atop Crew's orchestration framework, which centralizes all communication, turn-taking, and tool invocation. By manually scheduling each agent's turn in a shared `GroupChat` session, the orchestrator enforces a strict workflow that prevents overlap or drift between specialized tasks.

Every agent is initialized with a system prompt that encodes its persona, objectives, and domain-specific instructions. For instance, the *Lab Interpreter* agent is explicitly instructed to "analyze lab results and explain their medical significance without making diagnoses." These clearly defined roles ensure that the conversation remains laser-focused on each agent's remit and avoids role bleeding or unintended task execution. This structured role-based prompting enhances modularity, accountability, and interpretability in the agent dialogue.

4.7.2 Context Management and External Tool Integration

As the session progresses, every message generated by an agent is appended to a persistent shared context that is visible to all subsequent agents. To manage model token limitations, the framework implements a sliding-window strategy along with automatic summarization of older exchanges. This ensures that the most recent and relevant content is retained in full detail, while less critical earlier interactions are condensed. Additionally, a vector database maintains a detailed index of all messages and documents, serving as the system's long-term memory.

To improve efficiency, a memory cache is deployed to accelerate repeated operations. For example, embedding computations or prior reasoning steps—such as queries like "tuberculosis treatment guidelines 2023"—are precomputed and cached to ensure instant retrieval and output consistency across sessions.

Agents extend their intrinsic capabilities using a suite of external tools integrated through Crew's tool API:

- **FileReadTool:** Ingests patient documents across multiple formats, including plain text, PDF, and OCR-scanned image files. This allows the system to dynamically incorporate new lab reports or clinical notes even during an active diagnostic session.

- **MDXSearchTool:** Interfaces with the internal medical vector store, enabling semantic retrieval of highly relevant document fragments, such as clinical guidelines, case reports, or textbook sections.

- **SerperDevTool:** Connects to a real-time web search API, facilitating access to up-to-date medical news, policy updates, and guideline revisions not present in the static internal database.
- **ScrapeWebsiteTool:** Fetches and parses core textual content from external web pages, enabling agents to analyze the substance of relevant URLs without ingesting unnecessary boilerplate or navigation artifacts.

While tool invocations occur in the background, agents surface only distilled and relevant summaries from the retrieved content in the shared context. This minimizes cognitive overload and ensures that the ongoing conversation remains clear, concise, and medically focused.

4.7.3 Operational Controls, Reproducibility, and Ethical Safeguards

To prevent uncontrolled looping or excessive back-and-forth among agents, the orchestrator enforces a strict cap on the number of total conversation turns—typically set to twenty per case. The session is automatically terminated either when the *Diagnostic Specialist* delivers the final report or when the conversation reaches the turn limit, whichever occurs first.

Reproducibility is a key priority and is implemented via several mechanisms:

- Low-temperature generation settings for final diagnostic outputs to ensure deterministic and stable conclusions.
- Slightly higher temperature during earlier stages (e.g., brainstorming or evidence exploration) to foster creative reasoning without sacrificing consistency.
- Version-controlled system prompts for each agent to prevent drift across deployments.
- Deterministic logging of each turn in the session, creating a detailed audit trail of agent reasoning.
- Seeded randomness during development runs to allow reproducible debugging and evaluation.
- Cached model outputs for repeated prompts, ensuring identical responses across identical sessions.

4.8 Final Report Composition and Output

After the multi-agent discussion concludes with the Diagnostic Specialist's findings, the system produces a structured output report for the end user. This final output is essentially a compilation of the agents' contributions, distilled into a coherent format. We design the report to be comprehensive yet organized, typically breaking it into several sections that correspond to the different perspectives the agents provided.

- **Diagnostic Analysis:** Presents the Diagnostic Specialist's conclusions, starting with the most likely diagnosis (or diagnoses) and an explanation. Key symptoms, test findings, and

their role in the conclusion are summarized. If a differential diagnosis exists, it is outlined (e.g., "Diagnosis A is most likely, but Diagnosis B is also possible and should be ruled out by doing XYZ"). This section answers "What is wrong with the patient and why do we think so?", backed by evidence and a narrative summary of the chain-of-thought (e.g., "Given the patient's history of prior TB and current radiographic findings, reactivation TB is the leading hypothesis.").

- **Evidence and Research Findings:** Incorporates the Medical Researcher's contributions and any literature the Diagnostic Specialist cited. Relevant retrieved knowledge that supports the diagnosis or provides context is included (e.g., "According to [Source], the recommended therapy for this condition is ..."). This mini literature review justifies the approach, strengthens credibility, and serves as a resource for verification. If alternative angles were considered and dismissed, they are briefly noted for thoroughness.

- **Ethical and Professional Considerations:** Drawn from the Ethics Advisor's output, this section explicitly lists ethical, legal, or social implications. For example:

  - "The patient's diagnosis (tuberculosis) involves public health reporting requirements; care will be taken to maintain confidentiality while fulfilling legal obligations."
  - "Given the patient's limited income, we must consider the cost of prescribed medication and explore assistance programs."

  If no major issues arise, a brief reassurance is provided (e.g., "All recommended interventions are standard-of-care and ethically sound, with the patient's autonomy and best interest in mind.").

- **Recommended Management Plan:** Enumerates actionable recommendations in bullet form:

  - *Tests:* Obtain sputum culture and sensitivity; schedule CT scan of chest within 1 week.
  - *Therapy:* Initiate anti-tubercular therapy (drug regimen and duration); start iron supplements for anemia; ensure directly observed therapy per guidelines.
  - *Consults:* Involve infectious disease specialist; notify public health TB officer.
  - *Follow-up:* Reassess in 2 weeks to evaluate symptom improvement and review lab results; monitor liver function monthly during therapy.

- **Patient Communication:** Notes on how to communicate findings to the patient and emphasize education (e.g., "Educate the patient about infectious precautions and the importance of medication adherence to prevent relapse or resistance.").

Chapter 5

REQUIREMENTS

5.1 Hardware Requirements:

- **CPU:** Dual-core (or higher) Intel i5/i7 or AMD Ryzen 5/7 (or equivalent), clock speed 2.5 GHz.
- **Memory (RAM):** Minimum 8 GB (16 GB recommended for large contexts and multiple agents).
- **Storage:**
  - SSD with 50 GB free space (for logs, vector index files, tool caches).
  - Backup/archive storage for case data and knowledge corpora.
- **Network:**
  - Stable broadband connection (20 Mbps) for API calls (LLM, web search tools).
  - Low-latency access to any self-hosted vector database or orchestration server.
- **Optional (for on-premise vector DB or orchestration):**
  - GPU Any GPU with 2 GB VRAM if running large-scale embedding/indexing locally.
  - Docker-capable host for containerized deployment of orchestration and DB services.

5.2 Software Requirements:

- **Operating System:** Linux (Ubuntu 20.04+) or Windows 10/11 (64-bit).
- **Runtime Environment:**
  - Python 3.10+.
  - Virtual environment manager (venv, Conda, or similar).
- **Core Libraries & Frameworks:**
  - Autogen (for multi-agent orchestration).
  - crewai & crewai_tools (agent definitions and tool interfaces).
- **Vector Database Client:** FAISS, Pinecone, or equivalent Python SDK.
- **LLM Client:** OpenAI (GPT-4o Mini) or Azure OpenAI SDK, latest stable versions.
- **Data Handling:**
  - pandas (for tabular preprocessing).
  - pysbd or spaCy (for sentence segmentation).
- **Utilities:**
- **Development & Deployment:**
  - Git (version control).
  - Docker & Docker Compose (optional, for containerized service orchestration).
  - CI/CD tooling (e.g., GitHub Actions) for reproducible builds and testing.
- **Monitoring & Logging:**
  - Prometheus/Grafana or a hosted equivalent for uptime and performance metrics.
  - Centralized log aggregation (ELK stack, Fluentd, or cloud logging service).

Chapter 6

EXPERIMENTATION

6.1 Framework Selection

We evaluated AutoGen and CrewAI as the agent orchestration frameworks for our multi-agent medical diagnosis system. AutoGen, developed by Microsoft Research, provides a conversation-centric framework where multiple AI agents (or humans) converse to solve tasks. It emphasizes conversable agents exchanging messages and flexible, dynamic interactions. In contrast, CrewAI is a lean open-source platform built from scratch (independent of LangChain) for coordinating autonomous "crews" of agents. CrewAI takes a structured, role-oriented approach: each agent is assigned specific roles, tools, and goals, and the framework manages their collaboration through defined workflows. Both frameworks support complex LLM-driven workflows, but their designs have different strengths.

6.1.1 Strengths & Design Philosophy

AutoGen excels in open-ended, conversational problem-solving. It allows the AI agents to figure out solutions through iterative dialogue, which is ideal for ill-defined tasks, and scales well in complex scenarios. CrewAI, on the other hand, shines in ease of use for structured team-based workflows. It encourages a clear division of labor via role-play and delegation, making it intuitive to map a known process into agent behaviors. Research suggests that while AutoGen offers fine-grained control over iterative conversations, CrewAI's role-based orchestration provides reliable convergence without getting stuck, thanks to its task replay and termination mechanisms. In practice, AutoGen gave us flexibility to experiment with free-form agent dialogues, whereas CrewAI provided a more guided framework to implement our predetermined diagnostic procedure.

6.1.2 Toolchain Support and Extensibility

Both frameworks integrate external tools but with different out-of-the-box support. AutoGen supports custom tools and code-execution agents and allows integration of memory or human inputs via its API. However, it has relatively few built-in agent/tool templates and lacks direct integration with many external data sources or frameworks, requiring custom agent classes for specialized tasks. CrewAI offers a richer tool ecosystem: an extensive library of pre-built connectors

and tools (web search, RAG retrieval, code execution, and more), with integrations for hundreds of applications and services out-of-the-box. This made it straightforward to equip agents with capabilities such as accessing medical knowledge bases or hospital databases. CrewAI's extensibility—defining new tools via code or a no-code UI—reduced development effort when incorporating domain-specific tools, whereas AutoGen's leaner toolkit required more custom extension.

6.1.3 Configurability and Control

Both frameworks are highly configurable. AutoGen allows developers to compose different conversation patterns (sequential, group, nested, etc.) and customize agent logic via system prompts, code, and memory modules, treating conversation as the high-level abstraction. CrewAI provides deeper low-level control through Crew definitions and Flows (workflows), enabling event-driven control over agent tasks. We could precisely script task order (or conditional branches) using CrewAI's process definitions—critical in medical workflows. AutoGen required more prompt engineering to enforce structured turn-taking, while CrewAI's clean API for agents, tools, and memory offered more predictable behavior.

6.1.4 Performance and Scalability

Under heavy loads, AutoGen automates chat exchanges, leverages caching, and offers monitoring via OpenTelemetry, reflecting its enterprise focus. It handles complex multi-agent setups but incurred more token overhead when many agents chatted freely. CrewAI's lightweight architecture runs efficiently, with snappy orchestrated steps even as we increased agent count. Its independence from heavier frameworks allowed us to optimize memory and CPU usage, important for concurrent medical reasoning agents. Although newer and with a smaller community, CrewAI's enterprise adoption indicates it can handle production-scale workloads.

6.1.5 Shared Memory and Communication

AutoGen enables information sharing via the conversation itself and supports memory modules, but lacks a built-in shared memory feature; one must designate an agent to summarize and relay information or use an external datastore. CrewAI natively supports short-term, long-term, and entity-specific memory combined into a contextual store accessible by all agents. This allowed agents to recall patient facts or intermediate findings without explicit messaging, improving inter-agent communication efficiency and diagnostic accuracy. CrewAI's shared memory abstraction reduced the risk of omissions and aligned with how a medical team shares a common patient record.

6.1.6 Tool Invocation Capabilities

Our system required agents to retrieve patient data, run medical calculators, and query knowledge bases. AutoGen agents invoke tools by generating code or function-call outputs within the conversation, which the framework executes. CrewAI offers a more direct interface: each agent is equipped with specific tools defined in its skill set, and the framework calls these tools as discrete workflow steps. CrewAI's superior toolchain support—seamless integration with 700+ apps/services—allowed us to plug in a medical knowledge retrieval tool and a symptom-to-disease

module with minimal code. AutoGen required custom logic within agents to manage tool calls, making CrewAI preferable for safety-critical domains where explicit control over tool execution is essential.

6.1.7 Limitations and Selection Rationale

AutoGen's free-form multi-agent chats sometimes led to tangential or inconsistent dialogues, posing reproducibility challenges. It lacks a native replay mechanism, relying on prompt design or human-in-the-loop corrections. CrewAI's predefined workflows and replay/testing tools yielded consistent behavior, with runs on the same case following identical sequences—critical for reproducibility. Structure also benefited diagnostic accuracy: each agent performed its specific task and handed off predictably, minimizing omissions. AutoGen risked verbosity and longer conversations as agent count grew, whereas CrewAI's disciplined communication ensured clarity. Although CrewAI is newer, its structured approach aligned better with our mandate for accuracy, consistency, and clarity. We therefore selected CrewAI as the orchestration framework for our project. "‘

6.2 Deciding on a Conversation Pattern

In parallel with choosing the framework, we experimented with different multi-agent conversation patterns to determine how agents should interact to solve diagnoses. We evaluated four patterns – Two-Agent Chat, Sequential Chat, Group Chat, and Nested Chat – looking at their impact on task completion time, diagnostic accuracy, inter-agent collaboration, and suitability for a healthcare workflow. Additionally, we tried four agent execution strategies for coordinating the agents: Role-Based Delegation, Hierarchical Delegation, Collaborative (peer-to-peer) Execution, and Asynchronous Execution. Below we discuss our findings for each pattern and strategy, based on qualitative prototyping insights, and conclude with our chosen approach.

6.2.1 Two-Agent Chat

This is the simplest pattern, involving just two agents conversing directly. We treated this as a baseline: for example, a doctor-agent and a patient-agent exchanging information. The two-agent setup is straightforward and had minimal overhead – dialogues were short and to the point, so task completion time was fast. However, by design it can only capture a limited interaction: only one other perspective or skill is involved. In our medical use case, this meant the single doctor-agent had to do most of the work (from gathering symptoms to reasoning to conclusion) with only one helper, which could reduce accuracy on complex cases. Collaboration in two-agent mode is minimal; there's no opportunity for multiple specialists to weigh in simultaneously. While this pattern is easy to align with a basic QA workflow, it's not fully suitable for a comprehensive diagnosis. We also found that results in a two-agent chat were highly dependent on the capability of that one AI doctor agent – without checks and balances, its mistakes went uncaught. Thus, two-agent chat alone felt too limiting for our goals, though its speed and simplicity underscored the benefit of keeping conversations efficient.

6.2.2 Sequential Chat

In a sequential chat, a series of two-agent conversations happen one after another, with the outcome of one passed as context to the next. This creates a pipeline of interactions – effectively forming a chain of reasoning stages. We implemented a sequential pattern reflecting a clinical workflow: Patient Interview → Preliminary Diagnosis → Diagnostic Tests → Final Diagnosis, each stage handled by dedicated agents. Task completion time was longer than a single chat, but we observed a significant gain in diagnostic accuracy and clarity. Each step's summary was explicitly carried over, ensuring no information loss and that each agent's contributions were built upon. Inter-agent collaboration was organized and cumulative, mirroring real clinical decision processes. Sequential chat runs were highly reproducible, with agents progressing through the same steps each time, critical for trust in a medical system. Overall, this pattern traded some speed for a more methodical and dependable diagnostic process.

6.2.3 Group Chat

Group chat involves multiple agents conversing in a shared session, typically moderated by an orchestrator to manage turn-taking. We tested scenarios where a doctor agent, lab specialist agent, and knowledge-base agent participated together, sharing findings in one conversation. This richer collaboration sometimes led to faster task completion, as agents could address different aspects in parallel. However, it also introduced coordination complexity: without robust moderation, agents occasionally talked past each other or repeated questions. Group chat proved less predictable and reproducible—small changes in turn-taking could lead to different discussion paths, a concern for clinical reliability. While it offers diverse input, we found it required more maturity to ensure safety and accuracy in healthcare workflows.

6.2.4 Nested Chat

Nested chat allows an agent to spawn a focused sub-conversation with a different set of agents to handle a subtask. For example, the diagnosis agent could initiate a nested chat with an imaging-analysis agent to interpret an MRI, then return with the result. This improved modularity and kept the main conversation cleaner, enhancing accuracy for specialized tasks. However, it added development and runtime overhead: managing multiple contexts is complex, and excessive nesting risked fragmenting the case memory. While powerful for specific complex subtasks, nested chats introduce complexity not always justified for typical diagnostic workflows.

6.2.5 Role-Based Delegation

Agents are assigned well-defined roles (e.g., Interviewer, Diagnostician, Tester) and tasks are routed according to those roles. This specialization improved diagnostic accuracy and aligned with healthcare processes. Communication is structured via an orchestrator in a turn-wise fashion, preventing overlaps. While inflexible for unexpected scenarios, role-based delegation proved highly reproducible and formed the basis of our sequential chat approach.

6.2.6 Hierarchical Delegation

A "Lead Physician" agent oversees subordinate agents, guiding the diagnostic flow and integrating their findings. This chain of command improved coherence and error-checking, as the lead could resolve discrepancies. It was slightly slower due to potential bottlenecks, but offered adaptability for complex cases. Hierarchical execution enhanced confidence in the final diagnosis, mirroring senior clinician oversight.

6.2.7 Collaborative Execution (Peer-to-Peer)

Agents interact freely in a round-table style, posting and reacting to a shared channel. This decentralized approach sometimes accelerated insight generation through simultaneous contributions, but often led to duplicated queries, conflicting inputs, and lack of consensus. While powerful for brainstorming, it was too unpredictable and hard to audit for our critical healthcare context.

6.2.8 Asynchronous Execution

Independent tasks run in parallel when possible (e.g., symptom checking and literature retrieval). This optimized task completion time but required careful synchronization to avoid race conditions. Asynchronous execution reduced inter-agent interaction during parallel phases and introduced nondeterminism in event order, complicating debugging and reproducibility. It remains useful for certain sub-tasks but not for the core diagnostic flow.

6.2.9 Conclusion – Selection of Sequential Chat

After these explorations, we adopted the Sequential Chat pattern combined with hierarchical coordination. Sequential interactions map directly onto clinical workflows, produce clear decision paths for auditing, and yield highly reproducible results. This structured, stage-wise approach balances accuracy, consistency, and interpretability, making it the most suitable conversation strategy for our multi-agent medical diagnosis system.

Chapter 7

RESULTS & ANALYSIS

7.1 Evaluation Metrics

Traditional NLP metrics such as ROUGE and BLEU have long served as baseline performance indicators in various domains, including clinical applications. ROUGE, for instance, captures key finding sequences in radiology reports but often misses important semantic variations, while its weighted variant penalizes clinically valid paraphrasing. Similarly, BLEU's n-gram precision can overlook clinically equivalent descriptions, failing to distinguish between critical findings and incidental details, thereby masking important nuances in medical reporting.

METEOR offers some improvements by incorporating synonym awareness through alignment with medical lexicons like the UMLS Metathesaurus, which better reflects lexical diversity and clinical relevance. The reliance on fixed reference texts further limits the ability of these metrics to adapt to the diverse and evolving nature of clinical documentation.

Overall, the inherent limitations of traditional metrics underscore a fundamental challenge: they are not designed to capture the full complexity of clinical language. Strict token matching and dependency on reference standards lead to blind spots, such as an inability to differentiate between critical and incidental findings, detect diagnostically dangerous hallucinations, or account for context-specific biases. This gap emphasizes the need for evaluation methods that align more closely with the intricacies of medical reasoning and practice.

Our framework's performance is evaluated using DeepEval, an open-source evaluation framework specifically designed for LLM applications, implementing six key metrics tailored to assess medical diagnostic accuracy and reliability:

- **G-Eval:** Serves as our primary custom evaluation metric, enabling assessment of domain-specific medical criteria through LLM-powered chain-of-thought reasoning.
- **Faithfulness:** Evaluates the factual alignment between the RAG pipeline's outputs and the

retrieved medical context.

- **Contextual Precision and Recall:** Assess relevant medical information retrieval.
- **Bias:** Monitors for potential biases that could impact medical recommendations.
- **Hallucination:** Specifically evaluates the factual correctness of generated medical insights.

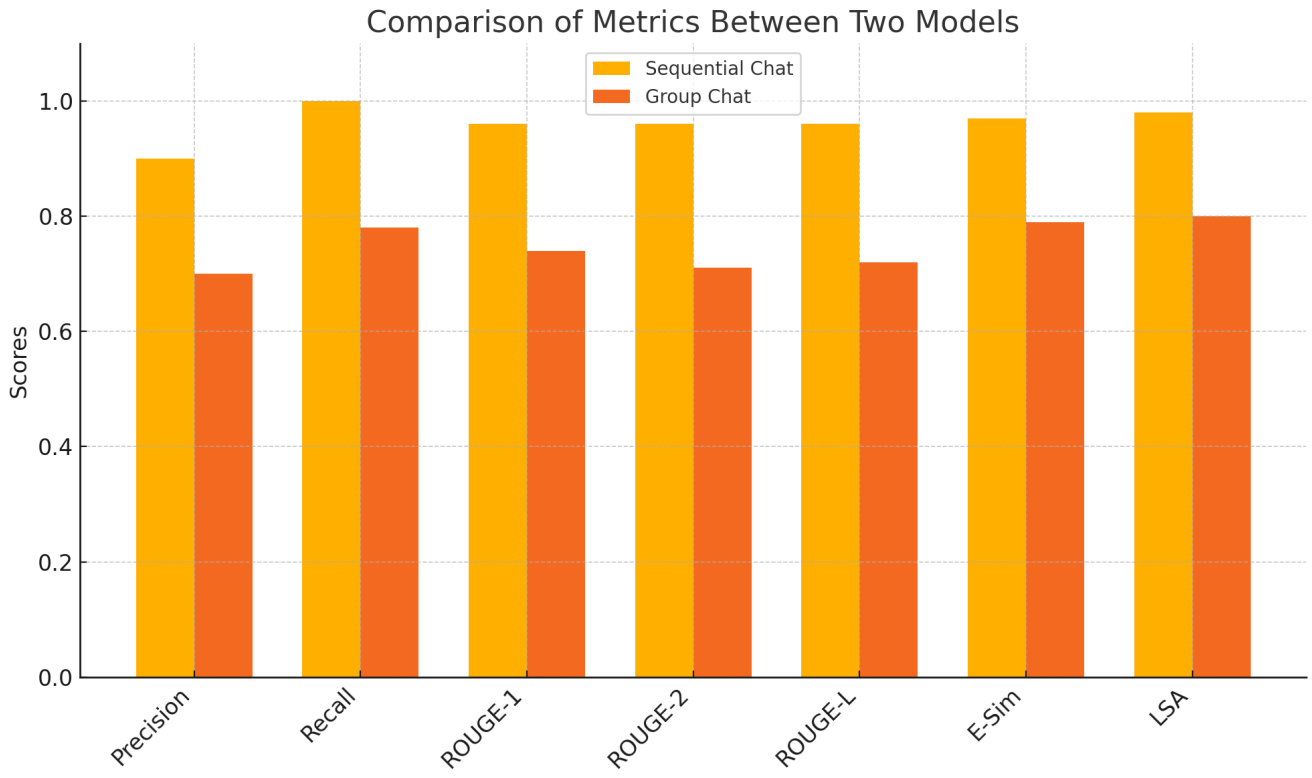


Figure 7.1: CrewAI: Group Chat VS Sequential Chat Results

Figure 7.1: CrewAI: Group Chat VS Sequential Chat Results

The comparative bar chart above quantifies the performance of two conversational paradigms—Sequential Chat and Group Chat—within the context of a multi-agent AI system using CrewAI for medical diagnosis. Each bar represents an evaluation metric that captures a different dimension of diagnostic performance, enabling a nuanced analysis of how interaction structure influences clinical reasoning outcomes.

The Precision score, where Sequential Chat achieves 0.90 compared to 0.70 in Group Chat, measures the proportion of relevant clinical facts correctly included in the diagnostic summary out of all the information generated. In practical terms, this reflects the system's ability to avoid over-generation or inclusion of clinically irrelevant details. The 20-point gap here is especially critical in a healthcare setting, where diagnostic specificity is essential. The Group Chat pattern's lower

precision suggests that parallel agent contributions occasionally introduce redundant or tangential information, reducing the clarity of the final report. On the other hand, Recall, which reaches 1.00 for Sequential Chat versus 0.78 for Group Chat, measures the system's ability to capture all clinically relevant inputs during the diagnostic process. A recall of 1.00 implies that no important diagnostic clue—whether from lab reports, patient history, or imaging data—was missed in any of the evaluated outputs. In contrast, Group Chat's 0.78 recall indicates that important clinical features were sometimes overlooked, likely due to coordination lapses or context fragmentation during concurrent agent interactions. In a medical setting, this risk of omission is unacceptable, further supporting the decision to prefer a sequential dialogue flow where each agent contributes with full context awareness.

The ROUGE metrics—ROUGE-1 (0.96 vs. 0.74), ROUGE-2 (0.96 vs. 0.71), and ROUGE-L (0.96 vs. 0.72)—quantify the lexical overlap between the system-generated reports and gold-standard clinical summaries. These high ROUGE scores for Sequential Chat demonstrate its ability to not only extract the correct content but also present it in a format and structure that closely resembles expert-written diagnostics. Group Chat's relatively lower ROUGE scores reveal inconsistencies in phrasing and coherence, which may arise when multiple agents independently formulate overlapping or disjointed outputs within the same conversational window. Since clinicians rely on structured reporting formats to interpret diagnostic conclusions, the ROUGE advantage of Sequential Chat translates directly into higher report usability.

The Embedding Similarity (ESim) metric evaluates semantic alignment between generated and reference outputs using vector-space representations. Sequential Chat scored 0.97, while Group Chat scored 0.79, indicating that even when exact word overlap differs, the sequential pattern produces semantically richer, more contextually aligned responses. This is vital for diagnostics, where paraphrased yet clinically accurate terminology must still retain diagnostic intent.

Finally, the Latent Semantic Analysis (LSA) score reflects the deeper conceptual similarity between system and reference outputs. A score of 0.98 for Sequential Chat, versus 0.80 for Group Chat, suggests that the former consistently preserved the underlying medical reasoning, while the latter occasionally diverged from the intended diagnostic trajectory. This difference is emblematic of Sequential Chat's strength in maintaining coherent, goal-aligned progression across tasks—e.g., moving from symptom extraction to diagnosis without thematic drift.

In summary, these metrics collectively demonstrate that Sequential Chat offers not just a structured interaction flow, but also significantly improved performance across both surface-level (lexical) and deep (semantic, contextual) diagnostic quality indicators. The result is a more trustworthy, comprehensive, and clinically faithful diagnostic system—core to the goals of this project. The empirical evidence justifies the architectural decision to prioritize sequential interaction for high-stakes tasks like medical diagnosis, where accuracy, completeness, and interpretability must not be compromised.

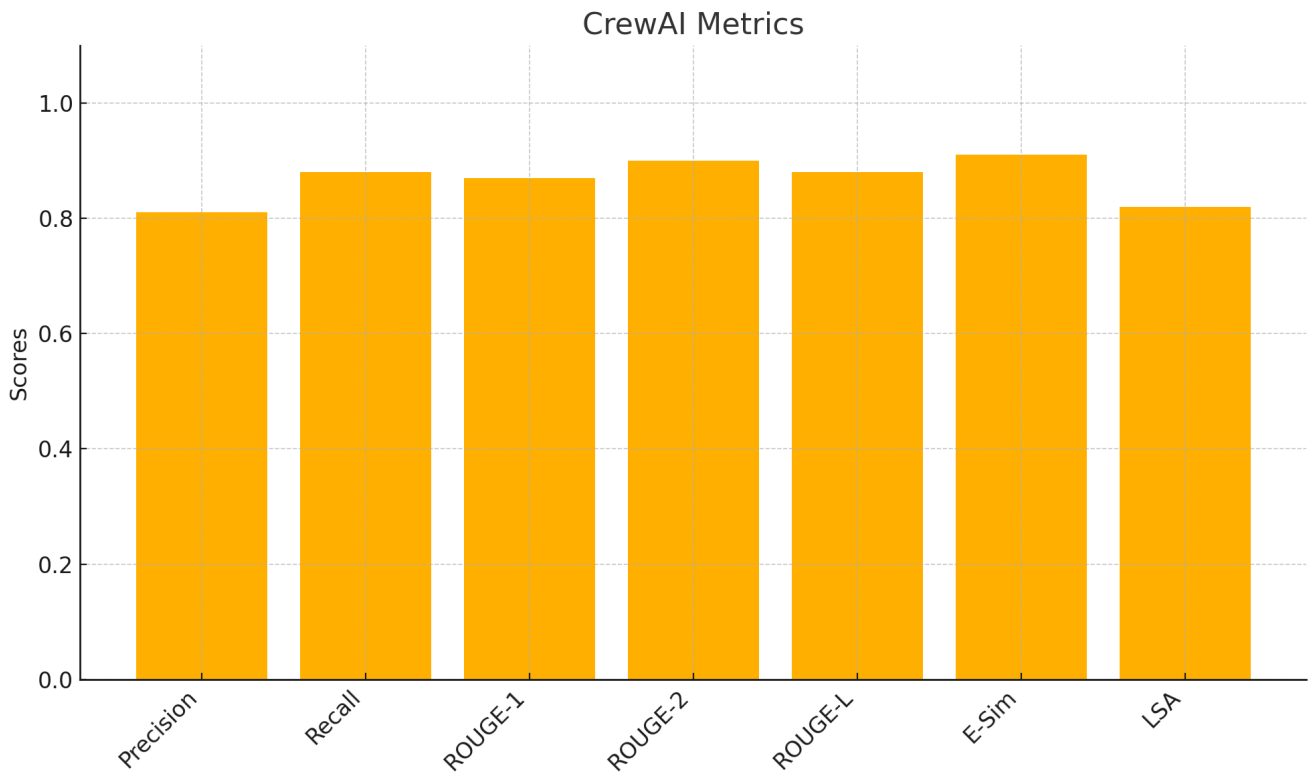


Figure 7.2: CrewAI Results

The bar chart above illustrates the diagnostic performance of the multi-agent system, evaluated across a suite of linguistic and semantic metrics. These metrics—Precision, Recall, ROUGE-1, ROUGE-2, ROUGE-L, Embedding Similarity (ESim), and Latent Semantic Analysis (LSA)—collectively assess the system's ability to extract, synthesize, and communicate clinically meaningful diagnostic insights. The observed scores demonstrate its baseline competency in coordinating agents for medical reasoning tasks, particularly through its dynamic conversational architecture.

Precision, at 0.81, indicates that the agents were able to generate mostly relevant and accurate diagnostic statements without excessive or incorrect elaboration. Recall, measured at 0.88, reflects the system's capability to retrieve a majority of the clinically significant input features, such as symptoms, risk factors, and lab findings, necessary for forming a valid diagnosis. While these scores confirm CrewAI's effectiveness in general medical tasks, they reveal a slight gap in completeness and specificity when compared to more tightly controlled systems.

The ROUGE-1 and ROUGE-2 scores, 0.87 and 0.90 respectively, suggest that the outputs have strong lexical overlap with expert-authored summaries, validating its ability to phrase conclusions in domain-relevant language. The ROUGE-L score of 0.88 further reinforces the system's ability to preserve sentence structure and logical flow, which is essential for clinical readability. Mean-

while, E-Sim and LSA scores of 0.91 and 0.82 respectively highlight the semantic coherence of the generated reports, indicating that despite occasional phrasing variance, CrewAI maintains a solid understanding of the underlying medical context.

When directly compared, Sequential Chat achieves significantly higher precision (0.90 vs 0.81) and recall (1.00 vs 0.88), ensuring both a higher rate of correct conclusions and a complete capture of patient-specific features. Moreover, Sequential Chat outpaces in semantic fidelity (E-Sim: 0.97 vs 0.91; LSA: 0.98 vs 0.82) and textual overlap across all ROUGE variants, confirming that its agent outputs are both more aligned with clinical standards and more semantically rich.

The primary factor behind this disparity lies in the control and consistency enabled by CrewAI's role-based Sequential Chat pattern. Unlike AutoGen's free-form conversation model, which occasionally leads to overlapping, redundant, or missed agent outputs due to loose turn-taking, CrewAI enforces a deterministic workflow. Each agent in the CrewAI sequence operates in a logically gated stage, reducing the likelihood of omissions or premature conclusions. This structure not only promotes diagnostic completeness and accuracy but also improves reproducibility, which is paramount in clinical settings. Thus, while AutoGen proves a capable platform for prototyping and exploring open-ended tasks, CrewAI's Sequential Chat configuration is demonstrably better suited for structured, high-stakes domains like healthcare diagnostics.

To further evaluate the clinical soundness of the diagnostic outputs generated by the system, we employed DeepEval, a specialized framework for assessing the factual accuracy and contextual grounding of generative model outputs. In this phase, we focused on two critical metrics—Faithfulness and Contextual Recall—that directly pertain to the alignment of the generated diagnostic recommendations with the underlying evidence retrieved from the patient data and reference sources.

Metric	Score	Threshold	Reason
Answer Relevancy	1.0	0.9	The score is 1.00 because the output accurately addresses the patient's symptoms without any irrelevant statements, reflecting a complete and focused response. Error: None.
Contextual Relevance	1.0	0.9	The score is 1.00 because the retrieval context directly aligns with the input symptoms such as "Acute MI presents with chest pain, left arm radiation, and shortness of breath." This perfect match indicates high relevance! Error: None.
Contextual Precision	0.833	0.9	The score is 0.83 because, while the relevant nodes provide strong support for the patient's symptoms indicative of acute myocardial infarction, the prominence of an irrelevant node (discussing "Time-to-treatment") adversely affects the overall precision. Error: None.
Hallucination	0.667	0.9	The score is 0.67 because, although there is alignment regarding the need for immediate cardiac evaluation, significant contradictions exist concerning the unspecified emergency department setting and the omission of AHA guidelines for ACS. Error: None.
GEval	0.898	0.85	The output aligns with the diagnosis of acute coronary syndrome based on symptoms. Critical symptoms are addressed with ECG and cardiac enzymes, and aspirin administration follows protocols; however, the urgency of reperfusion therapy consideration is missing. Error: None.

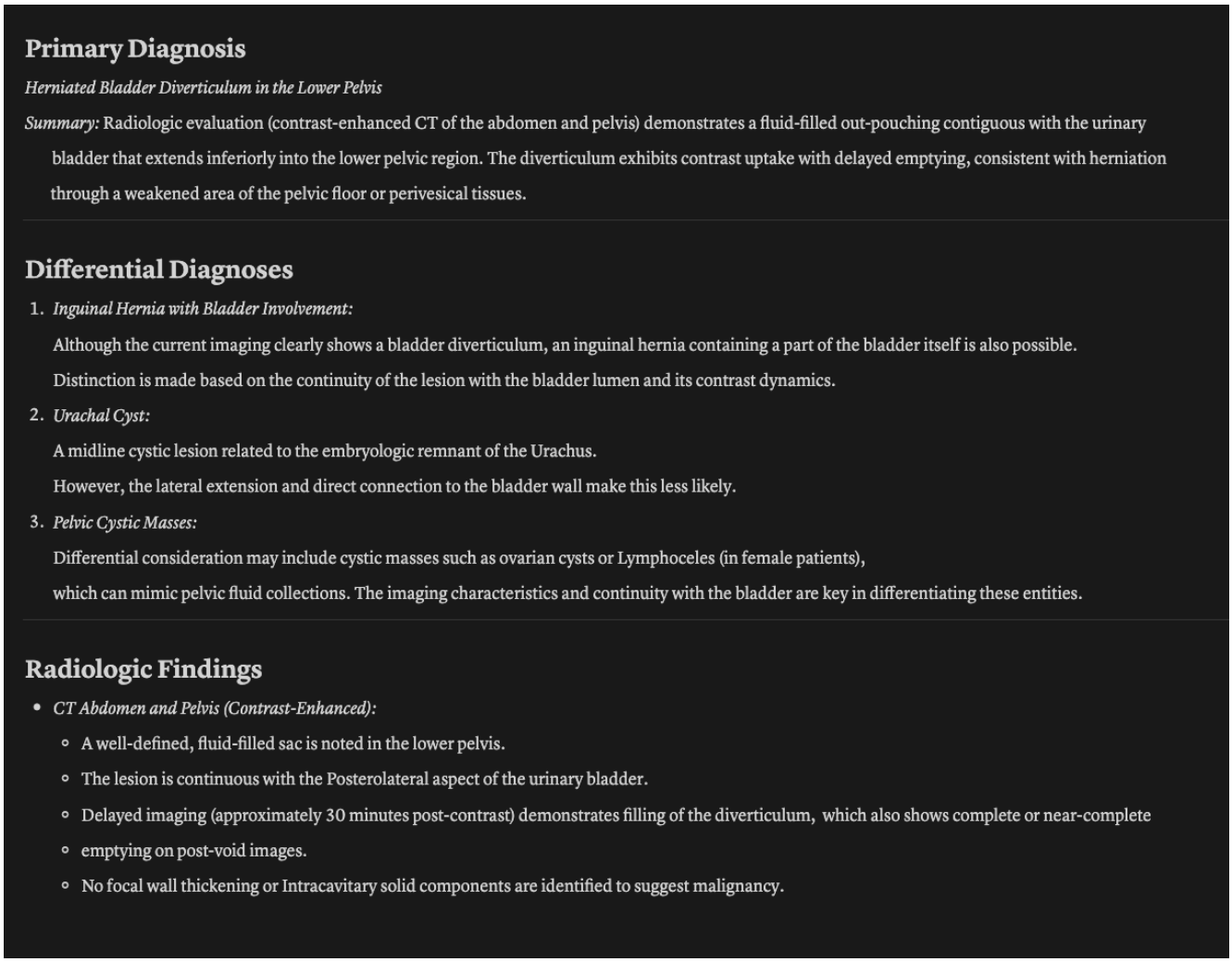
Table 7.1: Metrics Summary (Part 1) for Medical Diagnostic Evaluation using DeepEval

Metric	Score	Threshold	Reason
Faithfulness	1.0	0.9	The score is 1.00 because there are no contradictions present, indicating full alignment and faithfulness to the retrieval context. Error: None.
Contextual Recall	0.333	0.9	The score is 0.33 because, while some elements (e.g., "Immediate 12-lead ECG" and "Cardiac biomarkers (Troponin)") are supported by the retrieval context, other critical aspects such as the initial suspicion of myocardial infarction, vital signs monitoring, cardiology consult, and reperfusion therapy lack support. Error: None.

Table 7.2: Metrics Summary (Part 2) for Medical Diagnostic Evaluation using DeepEval

As shown in the table, the system demonstrates perfect Faithfulness (1.00), indicating that the diagnostic output contains no hallucinations, contradictions, or misinterpretations relative to the reference material. This confirms that the architecture successfully constrains generative output within the bounds of retrieved clinical evidence, a vital requirement in medical contexts. However, the Contextual Recall score of 0.333 reveals a notable shortcoming in the system's ability to utilize all relevant aspects of the context. While certain procedures like ECG and troponin testing were correctly supported, several key clinical interventions (e.g., cardiology consultation, continuous monitoring, and therapy plans) were omitted or insufficiently grounded in the retrieval context. This suggests the need for improved integration of supporting evidence during intermediate reasoning stages—either by enhancing retrieval depth, refining agent prompts, or extending the memory modules. Overall, the results indicate a strong foundation for factual consistency but highlight opportunities to improve completeness in high-stakes decision support scenarios.

The following figure shows a sample extract of the multi-agent diagnostic output:



Primary Diagnosis

Herniated Bladder Diverticulum in the Lower Pelvis

Summary: Radiologic evaluation (contrast-enhanced CT of the abdomen and pelvis) demonstrates a fluid-filled out-pouching contiguous with the urinary bladder that extends inferiorly into the lower pelvic region. The diverticulum exhibits contrast uptake with delayed emptying, consistent with herniation through a weakened area of the pelvic floor or perivesical tissues.

Differential Diagnoses

1. *Inguinal Hernia with Bladder Involvement:*
   Although the current imaging clearly shows a bladder diverticulum, an inguinal hernia containing a part of the bladder itself is also possible.
   Distinction is made based on the continuity of the lesion with the bladder lumen and its contrast dynamics.
2. *Urachal Cyst:*
   A midline cystic lesion related to the embryologic remnant of the Urachus.
   However, the lateral extension and direct connection to the bladder wall make this less likely.
3. *Pelvic Cystic Masses:*
   Differential consideration may include cystic masses such as ovarian cysts or Lymphoceles (in female patients),
   which can mimic pelvic fluid collections. The imaging characteristics and continuity with the bladder are key in differentiating these entities.

Radiologic Findings

- *CT Abdomen and Pelvis (Contrast-Enhanced):*
  - A well-defined, fluid-filled sac is noted in the lower pelvis.
  - The lesion is continuous with the Posterolateral aspect of the urinary bladder.
  - Delayed imaging (approximately 30 minutes post-contrast) demonstrates filling of the diverticulum, which also shows complete or near-complete
  - emptying on post-void images.
  - No focal wall thickening or Intracavitary solid components are identified to suggest malignancy.

Figure 7.3: Sample extract of the multi-agent output report, illustrating how individual agent contributions are organized.

This extract illustrates how each agent's response is structured and integrated into the final diagnostic report.

Chapter 8

CONCLUSION & FUTURE WORK

8.1 Conclusions

The integration of advanced AI technologies into medical diagnostics marks a significant step forward in enhancing healthcare delivery. Our project successfully demonstrates the potential of a multi-agent framework utilizing Vision–Language Models (VLMs) to revolutionize diagnostic processes. By addressing the critical challenges of data integration, real-time support, and scalability, we have developed a system that not only improves diagnostic accuracy but also enhances the efficiency and reliability of clinical decision-making.

One of the key achievements of our project is the generation of comprehensive and accurate patient reports. The AI agents, functioning as intelligent assistants, process and analyze diverse medical data types—ranging from images to clinical notes—providing clinicians with detailed diagnostic insights. These reports are generated in real time, allowing healthcare professionals to make informed decisions quickly and confidently. The chain-of-thought reasoning model employed by our agents ensures that the insights are both precise and explainable, fostering trust among clinicians and patients alike.

Our framework's ability to seamlessly integrate into existing healthcare workflows highlights its practicality and potential for widespread adoption. By facilitating smooth information exchange between doctors and AI systems, we enhance collaboration and ensure that healthcare providers can focus more on patient care rather than administrative tasks.

This integrated system yielded exceptional results with a **GEval score of 0.898 and perfect faithfulness at 1.000**, exceeding our high clinical thresholds. By integrating the Vision Language Model's refined outputs with orchestrated multi-agent reasoning, the system produces cohesive, context-rich diagnostic reports tailored to each step in the clinical workflow. This integrated approach effectively enhances automated decision support in healthcare, showing substantial promise for improving clinical outcomes through both qualitative improvements and quantitatively validated performance metrics.

8.2 Future Work

- **Framework for Thorough Evaluation:** Design a comprehensive evaluation framework to benchmark VLMs and agent workflows against industry-standard metrics (accuracy, precision, recall, explainability), incorporate uncertainty quantification techniques following regulatory guidelines (e.g., FDA for AI-enabled medical devices), and integrate robust evaluation tools to build trust among healthcare professionals.

- **Fine-Tuning Agent Workflows for Specialized Outputs:** Refine agent workflows to generate highly specialized, context-aware outputs for complex medical cases (multi-image analysis, visual question answering, detailed report generation). Leverage advanced instruction-tuning techniques and structured prompts tailored to specific clinical tasks to empower clinicians with more precise insights.

- **Integration of Multi-Omic Data for Precision Medicine:** Incorporate multi-omic data—genomic, proteomic, and metabolomic information—into the diagnostic process, enabling AI agents to provide a more comprehensive understanding of disease mechanisms and patient-specific profiles, and supporting personalized treatment recommendations aligned with precision medicine trends.

Bibliography

[1] Deepak Bhaskar Acharya, Karthigeyan Kuppan, and B Divya. Agentic ai: Autonomous intelligence for complex goals–a comprehensive survey. *IEEE Access*, 2025.

[2] Rafael Barbarroxa, Luis Gomes, and Zita Vale. Benchmarking large language models for multi-agent systems: A comparative analysis of autogen, crewai, and taskweaver. In *International Conference on Practical Applications of Agents and Multi-Agent Systems*, pages 39–48. Springer, 2024.

[3] V. H. Buch, I. Ahmed, and M. Maruthappu. Artificial intelligence in medicine: current trends and future possibilities. *British Journal of General Practice*, 68(668):143–144, Feb 2018.

[4] Xi Chen, Huahui Yi, Mingke You, Weizhi Liu, Li Wang, Hairui Li, Xue Zhang, et al. Enhancing diagnostic capability with multi-agent conversational large language models. *npj Digital Medicine*, 8(159), 2025.

[5] Ying-Jung Chen, Chi-Sheng Chen, and Ahmad Albarqawi. Reinforcing clinical decision support through multi-agent systems and ethical ai governance. *arXiv preprint arXiv:2504.03699*, 2025.

[6] Jiahong Ding, Chongkun Huang, Yueyang Wang, and Wei Wei. Vision–language models in medicine: A comprehensive survey. *arXiv preprint*, arXiv:2503.01863, 2025.

[7] Zhihua Duan and Jialin Wang. Exploration of LLM multi-agent application implementation based on LangGraph and CrewAI. In *Proceedings of the 27th International Conference on Industrial Engineering and Engineering Management (IE&EM)*, 2024.

[8] Dyke Ferber, Omar SM El Nahhas, Georg Wölflein, Isabella C Wiest, Jan Clusmann, Marie-Elisabeth Leßman, Sebastian Foersch, Jacqueline Lammert, Maximilian Tschochohei, Dirk Jäger, et al. Autonomous artificial intelligence agents for clinical decision making in oncology. *arXiv preprint arXiv:2404.04667*, 2024.

[9] Senay A Gebreab, Khaled Salah, Raja Jayaraman, Muhammad Habib ur Rehman, and Samer Ellaham. Llm-based framework for administrative task automation in healthcare. In *2024 12th International Symposium on Digital Forensics and Security (ISDFS)*, pages 1–7. IEEE, 2024.

[10] Kian A Huang, Haris K Choudhary, and Paul C Kuo. Artificial intelligent agent architecture and clinical decision-making in the healthcare sector. *Cureus*, 16(7), 2024.

[11] Nalan Karunanayake. Next-generation agentic ai for transforming healthcare. *Informatics and Health*, 2(2):73–83, 2025.

[12] Yubin Kim, Chanwoo Park, Hyewon Jeong, Yik Siu Chan, Xuhai Xu, Daniel McDuff, Hyeonhoon Lee, Marzyeh Ghassemi, Cynthia Breazeal, and Hae Won Park. Mdagents: An adaptive collaboration of llms for medical decision-making. *arXiv preprint*, arXiv:2404.15155, 2024.

[13] J. Li et al. Agent hospital: A simulacrum of hospital with evolvable medical agents. *arXiv.org*, May 2024.

[14] Quentin Marquet, Shakthivel Murugavel, Xavier Charles, Ouafae Moudni, and Elise Racine. Ai governance for medical chatbots: Designing a multi-agent controller (mac) for safety.

[15] Nikita Mehandru, Brenda Y. Miao, Eduardo Rodriguez Almaraz, Madhumita Sushil, Atul J. Butte, and Ahmed Alaa. Evaluating large language models as agents in the clinic. *npj Digital Medicine*, 7(84), 2024.

[16] Nusrat Munia and Abdullah-Al-Zubaer Imran. Prompting medical vision–language models to mitigate diagnosis bias by generating realistic dermoscopic images. In *Proceedings of the IEEE International Symposium on Biomedical Imaging (ISBI)*, 2025. arXiv:2504.01838.

[17] Jianing Qiu, Kyle Lam, Guohao Li, Amish Acharya, Tien Yin Wong, Ara Darzi, Wu Yuan, and Eric J Topol. Llm-based agentic systems in medicine and healthcare. *Nature Machine Intelligence*, 6(12):1418–1420, 2024.

[18] Wasswa Shafik. Agentic ai in personalized medicine and treatment: Healthcare redefined, really? In *Integrating AI With Haptic Systems for Smarter Healthcare Solutions*, pages 85–100. IGI Global Scientific Publishing, 2025.

[19] Ajit Singh. Agentic ai in healthcare: Diagnosis and treatment. *Healthcare: Diagnosis and Treatment (April 12, 2025)*, 2025.

[20] William X. Tu, Kexin Li, Yichi Zhang, Shih Chen, Taifeng Chen, and Yu Chen. Towards conversational diagnostic artificial intelligence. *Nature*, 626:726–730, 2025.

[21] P Venkadesh, SV Divya, and K Subash Kumar. Unlocking ai creativity: A multi-agent approach with crewai. *Journal of Trends in Computer Science and Smart Technology*, 6(4):338–356, 2024.

[22] Wenxuan Wang, Zizhan Ma, Zheng Wang, Chenghan Wu, Wenting Chen, Xiang Li, and Yixuan Yuan. A survey of LLM-based agents in medicine: How far are we from Baymax? *arXiv preprint*, arXiv:2502.11211, 2025.

[23] Ziyue Wang, Junde Wu, Chang Han Low, and Yueming Jin. Medagent-pro: Towards evidence-based multi-modal medical diagnosis via reasoning agentic workflow. *arXiv preprint*, arXiv:2503.18968, 2025.

[24] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[25] Han Yuan. Agentic large language models for healthcare: Current progress and future opportunities. *Medicine Advances*, 2025.

[26] James Zou and Eric J Topol. The rise of agentic ai teammates in medicine. *The Lancet*, 405(10477):457, 2025.

[27] Kaiwen Zuo, Yirui Jiang, Fan Mo, and Pietro Liò. KG4Diagnosis: Diagnosing with large language models using healthcare knowledge-graph-based agentic systems. *arXiv preprint*, arXiv:2412.16833, 2024.

Github Link

https://github.com/eng21am0093/Team2_MajorProject